

AGENCYOS

Technical / API / Data Contract Specification

PDF 3 of 10 | Phase 1 Agent Workforce | Contracts • APIs • Events • Schemas • Security • Reliability

Purpose: Define the technical contract layer that connects the six already-created Phase 1 documents: Phase 1 flow, Sales Agent, Scheduler Agent, Quotation Master Agent, Coordination Agent and Orchestrator/Router Agent. This document converts their interactions into implementation-ready API, task, event, data, integration and validation contracts.

Source basis: AgencyOS API & Integration, AI Orchestrator/Routing, AI Agent Architecture, CRM/Sales, WhatsApp Communication, Admin Policy and Automation/Jobs specifications. The source requires adapter-based integrations, normalized provider responses, authenticated/idempotent webhooks, correlation IDs, safe retries, explicit permissions, external-vs-domain state separation and schema validation before workflow state changes.

■filecite■turn11file1■ ■filecite■turn11file12■

1. Contract Architecture

CLIENT/WEB/WHATSAPP → API/COMMUNICATION ADAPTER → AUTH/IDENTITY → EVENT/QUEUE → DOMAIN STATE → COORDINATION TASK → ORCHESTRATOR → CAPABILITY/POLICY → PROVIDER ADAPTER → MODEL/TOOL → RESPONSE
NORMALIZATION → VALIDATION → DOMAIN STATE → AUDIT

- External services are accessed through adapters/connectors.
- Provider-specific IDs never become the only AgencyOS identity.
- Internal domain state is separate from external provider state.
- Workflow state changes occur only after validation.
- Agent output is treated as untrusted until schema/business validation passes.
- All important side effects have traceability and idempotency.
- Policy and authorization are enforced by backend/application controls, not prompts alone.

2. Technical Contract Layers

- REST/API layer for synchronous application operations.
- Event layer for asynchronous workflow changes.
- Task layer for agent work assignment.
- Result layer for structured agent completion.
- Provider adapter layer for external APIs.
- Domain layer for authoritative CRM/quote/payment/project state.
- Audit layer for traceability.
- Job layer for delayed/retry/scheduled work.
- Policy layer for authorization and approval.
- Integration layer for credentials, health, rate limits and provider configuration.

3. Universal Identifiers

- organization_id: tenant boundary.
- client_id: stable client identity.
- lead_id: sales opportunity identity.
- project_id: delivery identity where applicable.
- task_id: unique execution/work item identity.
- event_id: unique event identity.
- correlation_id: end-to-end trace identity.
- approval_id: governed approval identity.
- quote_id: quotation identity.
- quote_version: immutable commercial version.
- requirement_version: confirmed requirement/scope identity.
- artifact_id/version: immutable document/design/build/etc. reference.
- provider_object_id: external object identity.

- idempotency_key: duplicate-side-effect protection.
- request_id: individual API request trace.

4. Universal Request Envelope — Recommended

- request_id
- correlation_id
- organization_id
- actor_id / actor_type
- timestamp
- api_version
- operation
- resource identifier
- payload
- authorization context
- data-classification metadata where needed
- idempotency_key for side-effecting operations
- client/project/lead scope where applicable

5. Universal Response Envelope — Recommended

- request_id
- correlation_id
- api_version
- status
- data
- warnings
- error object when failed
- domain_state_reference
- created/updated resource ID
- evidence/reference IDs where applicable

6. Canonical Agent Task Contract

- task_id
- organization_id
- client_id
- lead_id
- project_id where applicable
- source_agent
- destination_agent or capability
- task_type
- workflow_state
- intent
- context_references
- requirement_version
- artifact/version references
- acceptance_criteria
- required_output_schema
- policy_reference
- approval_reference
- priority

- deadline
- dependencies
- attempt_number
- retry_budget
- previous_failure_summary
- idempotency_key
- created_at / updated_at

7. Canonical Agent Result Contract

- task_id
- correlation_id
- agent_id/version
- result_status
- structured_result
- schema_version
- evidence_references
- artifact/version references
- warnings
- blocker
- error_class
- attempt_number
- provider/model metadata where available
- usage/cost metadata where available
- validation_status
- next_action
- completion_evidence

8. Task State Contract

Recommended: CREATED → VALIDATING → READY → DISPATCHED → ACKNOWLEDGED → IN_PROGRESS → WAITING_FOR_RESULT → RESULT_RECEIVED → VALIDATING_RESULT → ACCEPTED → HANDOFF_READY → HANDED_OFF → VERIFIED → CLOSED. Exceptions: BLOCKED, RETRYING, FAILED, EXPIRED, CANCELLED, ESCALATED.

- Backend controls state transitions.
- Each transition records timestamp and actor/system source.
- Invalid transitions return CONFLICT/validation error.
- Result receipt does not automatically mean task acceptance.
- Evidence is required for configured completion conditions.
- State is durable across process restarts.

9. Phase 1 Internal API Surface — Recommended

Method	Endpoint	Purpose
POST	/api/v1/agent-tasks	Create a structured agent task
GET	/api/v1/agent-tasks/{task_id}	Read task/status
POST	/api/v1/agent-tasks/{task_id}/dispatch	Dispatch after validation
POST	/api/v1/agent-tasks/{task_id}/result	Submit structured result
POST	/api/v1/agent-tasks/{task_id}/retry	Request safe retry
POST	/api/v1/agent-tasks/{task_id}/cancel	Cancel permitted task
GET	/api/v1/agent-tasks/{task_id}/trace	Execution/handoff trace
POST	/api/v1/events	Publish internal event
POST	/api/v1/webhooks/{provider}	Receive authenticated provider event

Method	Endpoint	Purpose
GET	/api/v1/quotes/{quote_id}	Read quotation/current version
POST	/api/v1/quotes	Create quote draft through authorized workflow
POST	/api/v1/approvals	Create governed approval request
POST	/api/v1/approvals/{approval_id}/decision	Record authenticated decision

10. Phase 1 Domain API Contracts

- Lead API: create/update/read lead with normalized contact data and ownership.
- Sales task API: create/return qualification, discovery, objection and next-action results.
- Scheduler API: availability/proposal/booking/reschedule/cancel/result references.
- Quotation API: create draft/version, validate, approve-reference, deliver-reference, negotiate/revise.
- Coordination API: create task, dependency, blocker, handoff, escalation and verification records.
- Orchestrator API: capability lookup, route decision, execution, result validation, retry/fallback.
- Approval API: create exact-action request, record decision, verify current validity.
- Audit API: append/read authorized trace events.

11. Agent Routing Contract

- Input specifies required capability/task type.
- Orchestrator resolves eligible agents from registry.
- Agent capability is checked.
- Permissions are checked.
- Data-classification restrictions are checked.
- Tool requirements are checked.
- Output schema compatibility is checked.
- Provider/model candidates are evaluated.
- Routing policy is applied.
- Selected agent/model/provider and policy version are recorded.
- Fallback route is recorded when configured.

Source-defined routing chain: AGENT TASK → ORCHESTRATOR → CAPABILITY MATCH → POLICY → PROVIDER HEALTH → COST/QUALITY → MODEL SELECTION → EXECUTION → OUTPUT VALIDATION → SUCCESS/FALLBACK → AUDIT → NEXT WORKFLOW STEP.

■filecite■turn11file6■

12. AI Provider Adapter Contract

- Provider name.
- Provider adapter version.
- API endpoint/reference.
- Authentication method.
- Model ID/version.
- Capabilities.
- Modalities.
- Context limits.
- Structured output support.
- Tool-calling support.
- Pricing metadata.
- Rate limits.
- Health check.
- Error normalization.
- Streaming/batch support where relevant.
- Privacy/data-policy metadata.

- Provider availability state.

Provider-specific behavior must remain inside adapters. Core business logic must use the normalized AgencyOS contract. Provider credentials are securely referenced and never exposed to agents unnecessarily. ■filecite■turn11file13■

13. Model Selection Contract

- Quality score.
- Policy fit.
- Availability.
- Reliability.
- Capability fit.
- Cost.
- Latency.
- Required model pin/version where configured.
- Canary/experimental eligibility.
- Agent-specific provider rules.
- Data-classification restrictions.
- Budget state.
- Admin enable/disable state.

Source routing formula: $\text{ROUTE_SCORE} = \text{Quality} \times \text{PolicyFit} \times \text{Availability} \times \text{Reliability} \times \text{CapabilityFit} \div (\text{Cost} \times \text{LatencyPenalty})$. Exact weights must be configurable/versioned rather than hard-coded. ■filecite■turn11file7■

14. Structured Output Contract

- Workflow-critical AI tasks must request a defined schema.
- Quotation output uses quotation schema.
- Requirement extraction uses requirement schema.
- Agent handoff uses task schema.
- Approval request uses approval schema.
- Finance result uses invoice/payment schema.
- JSON/schema validation happens before accepting output.
- Schema version is stored with task/result.
- Schema failure becomes a canonical error, not a success.

15. Context Contract

- Context authorization occurs before provider call.
- Only minimum relevant context is passed.
- Client/project/tenant boundaries are preserved.
- Exact requirement/scope/quote/artifact versions are referenced.
- Provenance is retained for important facts.
- Secrets are excluded.
- Unnecessary PII is minimized/redacted.
- Stale context is detected where material.
- Memory is retrieved through authorized context services.
- Provider fallback inherits the same organization/data policy.

16. Permission Contract

- Agent permission checked.
- Tool permission checked.
- Integration capability permission checked.
- Tenant permission checked.
- Actor authorization checked.

- High-risk capability requires policy check.
- Provider response cannot grant permissions.
- Model output cannot approve its own exception.
- Authorization failure is canonical and auditable.

API Integration defines capabilities such as SEND_MESSAGE, READ_MESSAGE, CREATE_GROUP, VERIFY_PAYMENT, CREATE_INVOICE, RUN_BUILD, DEPLOY_RELEASE and QUERY_AI_MODEL, with permission checks before provider calls.

17. Webhook Contract

- Provider event ID / equivalent key.
- Provider name.
- Signature/authentication evidence.
- Received timestamp.
- Raw/reference payload according to retention policy.
- Normalized event type.
- Internal mapped resource ID.
- Organization/project/lead/client scope.
- Processing status.
- Idempotency result.
- Correlation ID where derivable.
- Error/rejection reason.

Inbound WhatsApp contract: PROVIDER WEBHOOK → VERIFY SIGNATURE/AUTH → DEDUPE → IDENTIFY CONTACT/PROJECT → NORMALIZE MESSAGE → STORE → CLASSIFY → ROUTE → ACTION → RESPONSE → AUDIT.

18. External ID Mapping

- Map provider external ID to stable internal ID.
- Store provider and provider_object_id.
- Enforce uniqueness within provider/context.
- Normalize phone/email/date/timezone where appropriate.
- Do not duplicate authoritative records unnecessarily.
- Keep external and domain state separate.
- Use reconciliation for critical external state.

The API specification explicitly requires stable internal IDs, provider/object mapping, normalized dates/timezones and separate external versus AgencyOS domain state.

19. Idempotency Contract

- Webhook uses provider event ID/equivalent key.
- Agent side effects use idempotency key.
- Quote creation cannot duplicate on retry.
- Invoice/payment cannot duplicate on retry.
- Message send operations use safe idempotency where supported.
- Project creation cannot duplicate on retry.
- Idempotency records have operation-appropriate retention.
- Duplicate requests return/reconcile existing outcome.

The source explicitly states that agent retries must not create duplicate quotes, invoices, payments, messages or projects.

20. Retry / Timeout / Circuit Breaker Contract

- Every external call has an explicit timeout.
- Retry policy is operation-aware.
- Retry-After is respected where provider supplies it.

- Invalid credentials are not retried indefinitely.
- Permanent validation errors are not retried.
- Asynchronous retry state is persisted.
- Repeated provider failures can open a circuit.
- Health recovery can close the circuit.
- Fallback is policy-approved.
- Critical financial/destructive operations are never blindly replayed.

21. Canonical Error Model

Code	Meaning
AUTHENTICATION_ERROR	Provider/credential authentication failed
AUTHORIZATION_ERROR	Actor/agent/integration lacks permission
VALIDATION_ERROR	Request/result failed schema/business validation
RATE_LIMITED	Provider throttled request
TIMEOUT	Operation exceeded configured timeout
PROVIDER_UNAVAILABLE	Provider unavailable/outage
NOT_FOUND	Referenced external/internal object not found
CONFLICT/DUPLICATE	Conflict or duplicate/idempotency collision
PROVIDER_ERROR	Provider returned an operational error
NETWORK_ERROR	Network/connectivity failure
POLICY_BLOCKED	Policy forbids requested operation
UNKNOWN_ERROR	Unclassified failure requiring safe handling

22. Quote / Version Data Contract

- quote_id identifies the commercial proposal.
- quote_version identifies immutable material revision.
- client details are linked.
- project/service summary is linked.
- line items, quantity, rate and discount are structured.
- tax/GST data is structured.
- total is deterministic from stored values.
- payment schedule is structured.
- validity is stored.
- terms are version-specific.
- approval references exact quote version.
- delivery references exact quote version.
- acceptance references exact quote version.
- Historical versions remain immutable.

23. Approval Data Contract

- approval_id
- organization_id
- requesting_agent
- request_type
- client_id
- project_id
- task_id
- exact action requested

- input values
- amount/value
- artifact/version
- policy version evaluated
- risk level
- reason
- evidence
- expiration
- required Admin role
- status
- created_at
- decision_at
- decided_by
- decision comment

Approval states are DRAFT → SUBMITTED → UNDER_REVIEW →

APPROVED/REJECTED/CHANGES_REQUESTED/EXPIRED/CANCELLED → EXECUTED → VERIFIED. Approval is permission for a specific action under specific conditions, not blanket authority. ■filecite■turn11file8■

24. Communication / WhatsApp Data Contract

- message_id
- provider_message_id
- conversation/group reference
- organization/client/project/lead mapping
- direction
- message type
- original content/reference
- detected language/confidence where used
- sender/recipient
- template ID/version where applicable
- delivery status
- read status where supported
- timestamp
- correlation/task reference
- approval/request reference where applicable
- audit reference

WhatsApp messages are persisted and auditable; webhook events are idempotent; important decisions become structured records; payment proof is not automatically verified; approval messages reference exact versions/requests. ■filecite■turn11file0■

25. API Versioning & Backward Compatibility

- Internal APIs use versioning where breaking changes are possible.
- External provider versions are isolated inside adapters.
- Compatibility periods exist for critical integrations.
- Deprecations are documented.
- Webhook changes support migration.
- Schema versions are explicit.
- Provider API changes do not silently alter domain behavior.
- Breaking contract changes require controlled rollout/testing.

These requirements are directly specified in the API & Integration document. ■filecite■turn11file3■

26. Authentication & Authorization

- Authenticate API caller.
- Authorize operation against organization/tenant.
- Authorize resource scope.
- Authorize agent capability.
- Authorize integration capability.
- Require elevated authorization for high-risk actions.
- Do not use natural-language claims as authorization.
- Record important authorization decisions.
- Keep secrets in secure secret references.
- Mask secrets in Admin UI/logs.

27. Rate Limits & Resource Protection

- Per-provider throttling.
- Per-organization throttling.
- Agent loop protection.
- Priority for critical workflows.
- Queue non-urgent work when needed.
- Expose rate-limit state to routing.
- Prevent uncontrolled concurrency.
- Record rejected/throttled operations.

28. Observability Contract

- request_id
- correlation_id
- task_id
- event_id
- provider
- model
- agent
- endpoint/operation
- organization/project context where safe
- latency
- status
- retry count
- error class
- external request/event ID
- usage/cost metadata where available
- validation result

Never log raw secrets or unnecessary sensitive payloads. Integration audit events include provider/model selection, external messages, payment events and integration configuration changes where applicable. ■filecite■turn11file12■

29. Integration State vs Domain State

- Provider message status remains external state.
- AgencyOS communication task remains internal domain state.
- Payment provider status is evidence, not automatically verified payment.
- AI provider availability affects routing, not agent authorization.
- External conflicts do not blindly overwrite approved internal state.
- Financial/security/approval conflicts route to authorized review.

- Both internal and external evidence are preserved where required.

30. Contract Testing

- Mock provider contract tests.
- Provider sandbox tests.
- Webhook signature tests.
- Duplicate event tests.
- Retry tests.
- Timeout tests.
- Rate-limit tests.
- Invalid credential tests.
- Provider outage tests.
- Fallback tests.
- Tenant-isolation tests.
- Critical end-to-end tests.
- Request schema validation.
- Response normalization tests.
- API-version compatibility tests.

31. Admin Technical Configuration

- Provider selection.
- Secure credential references.
- Model activation.
- Agent-to-provider assignment.
- Capability mapping.
- Fallback order.
- Cost limits.
- Latency limits.
- Daily/monthly usage.
- Budget alerts.
- Routing logs.
- Manual override.
- Emergency provider disable.
- Configuration version/history.
- Webhook configuration.
- Retry policies.
- Health status.
- Test connection.

The Orchestrator specification requires an Admin AI Routing Control Center with provider/model activation, fallback order, agent-to-provider assignment, capability mapping, limits, usage, logs, manual override, emergency disable and configuration history. ■filecite■turn11file13■

32. Data Integrity Invariants

- Duplicate event cannot duplicate business effect.
- Errors cannot become zero/empty/success state.
- Accepted quote baseline cannot be silently edited.
- Approval applies to exact action/version.
- Payment verification cannot be inferred from message alone.
- WON requires configured evidence/gates.
- External provider success cannot override missing internal evidence.

- Cross-tenant IDs cannot resolve into another tenant.
- Idempotency prevents duplicate financial/communication/project effects.
- Every important workflow decision has audit evidence.

33. Phase 1 API Sequence Examples

- A. Lead:** CRM lead event → create task → Orchestrator route → Sales result → validation → Coordination state update.
- B. Scheduler:** meeting intent → Scheduler task → route → availability/booking → result → validation → CRM/Coordination update.
- C. Quote:** confirmed requirement version → quote task → route → Quotation Master → quote version → policy/approval → delivery.
- D. Close:** exact acceptance → payment/exception workflow → verification → WON transition → handoff.
- E. Failure:** provider error → normalize canonical error → retry/fallback if safe → otherwise escalation; never false success.

34. Technical Build Order

Foundation

- ID strategy
- Tenant scoping
- API versioning
- Auth/RBAC
- Error model
- Audit schema

Contracts

- Task schema
- Event schema
- Result schema
- Handoff schema
- Approval schema
- Quote/version schema

Infrastructure

- Event bus/queue
- Job system
- Idempotency store
- Retry manager
- Provider adapters
- Secret references

Phase 1 APIs

- Lead/CRM
- Agent tasks
- Scheduler
- Quotation
- Approval
- Communication
- Trace

Orchestrator

- Capability registry
- Routing policy
- Provider/model registry
- Health
- Fallback

- Output validation

Reliability

- Timeout
- Rate limit
- Circuit breaker
- Dead-letter
- Reconciliation
- Conflict handling

Testing

- Contract tests
- Webhook tests
- Idempotency
- Tenant isolation
- E2E

35. Technical Acceptance Checklist

- All API requests validate schema.
- All side-effect APIs support idempotency where required.
- All webhooks authenticate and deduplicate.
- External IDs map to stable internal IDs.
- Provider responses normalize to internal contracts.
- Agent tasks use correlation IDs.
- Agent outputs are schema-validated.
- Permissions are checked before tool/provider calls.
- Policy blocks cannot be bypassed by model output.
- Quote versions remain immutable.
- Approvals reference exact versions/actions.
- External state is separated from domain state.
- Retries are bounded and persisted.
- Timeouts and circuit breakers exist where required.
- Fallback preserves policy/data constraints.
- Secrets are protected.
- Audit records are queryable.
- Admin can configure/disable integrations.
- Provider failure cannot produce false success.
- Critical contracts have automated tests.

36. Definition of Done

- PDF 2 interaction contracts are fully represented as technical schemas/events.
- Phase 1 agents can exchange structured tasks/results.
- Orchestrator can route using capability/policy/provider health.
- Coordination can track task/handoff state.
- Scheduler and quotation interactions have versioned references.
- Approval and acceptance are exact-version controlled.
- Webhook/provider integrations are authenticated and idempotent.
- External/domain state separation is implemented.
- Retry/fallback/degradation are safe.
- Tenant isolation and permissions are enforced.

- Observability and audit are complete.
- Contract tests pass.
- Critical end-to-end Phase 1 scenario passes.
- No unresolved API/data contract ambiguity remains.

37. Final Technical Contract

ONE RULE: Every important AgencyOS action must be representable as a typed, authorized, traceable operation with a stable identity, explicit scope, validated input, deterministic/backend-controlled state transition, structured output/evidence and safe failure behavior.

Architecture: External provider → Adapter → Normalized contract → Authorization/Policy → Agent/Orchestrator execution → Validation → Domain state → Event/Audit. Provider availability can change routing, but provider output never becomes business authority by itself.